

Partie 1 : satisfiabilité, insatisfiabilité, résolution.

On rappelle

- qu’un ensemble de clauses est *insatisfiable* si, en appliquant la méthode de résolution un certain nombre de fois, en enrichissant chaque fois l’ensemble de départ avec les déductions qui sont faites, on arrive à obtenir $\perp$ (la clause vide) ;
- qu’un ensemble de clauses S a pour conséquence la clause A si l’ensemble $S \cup \{\neg A\}$ est insatisfaisable ;
- qu’un ensemble de clauses est *satisfiable* si, en appliquant la méthode de résolution un certain nombre de fois, en enrichissant chaque fois l’ensemble de départ avec les déductions qui sont faites, on arrive à saturation (c’est à dire qu’on ne produit pas de nouvelle clause).

Exercice 1. Utiliser la méthode de résolution pour répondre aux questions suivantes :

1. Prouver que l’ensemble de clauses $S = \{P, \neg P \vee Q, \neg Q \vee R, \neg Q \vee \neg R\}$ est insatisfiable.
2. L’ensemble de clauses $S = \{P \vee Q, P \vee \neg Q, R \vee Q, R \vee \neg Q\}$ est-il satisfiable ou insatisfiable ?
3. Montrer que l’ensemble de clauses $\{P \vee Q \vee R, \neg P \vee Q \vee R, \neg Q \vee R\}$ a pour conséquence R .

Exercice 2. Après l’avoir converti en un ensemble de clauses, en utilisant unification et résolution, montrer par réfutation que l’ensemble de formules

$$\begin{aligned} S = \{ & (\text{vivant}(X) \wedge \text{mange}(X, \text{viande})) \rightarrow \text{carnivore}(X), \\ & \neg \text{animal}(X) \rightarrow \text{vegetal}(X), \text{animal}(X) \vee \text{vegetal}(X) \rightarrow \text{vivant}(X), \\ & \text{mange}(\text{emilie}, \text{poisson}), \text{vivant}(\text{albert}), \neg \text{animal}(\text{dagobert}), \text{mange}(\text{dagobert}, \text{viande}) \} \end{aligned}$$

a pour conséquence $\text{carnivore}(\text{dagobert})$ (*suggestion stratégique* : partir de la clause à prouver et remonter dans l’arbre de démonstration).

Partie 2 : Prolog.

Utiliser votre éditeur préféré pour créer un fichier d’extension `.pl` dans lequel vous écrirez votre programme (par exemple `td3.pl`). Attention à ne pas utiliser de majuscule pour la première lettre du nom du fichier. A partir du répertoire dans lequel figure votre fichier, lancer l’interpréteur Prolog dans le terminal courant en tapant la commande `swipl`. Charger votre programme en tapant son nom entre crochets (par exemple `[td3].`) ; à chaque modification du programme, n’oubliez pas d’enregistrer et de recharger le fichier dans Prolog.

Exercice 3. A l’aide des prédicats associés, déclarer les connaissances suivantes permettant de définir les liens familiaux reliant les individus d’une même famille :

- Albert, Jean, Paul Bertrand, Louis et Benoît sont des hommes ;
- Marie, Germaine, Sophie, Christiane, Simone sont des femmes ;
- Louis est le père de Benoît, Albert est le père de Jean, Jean est le père de Simone et de Paul, Paul est le père de Sophie et de Bertrand ;

- Marie est la mère de Sophie et de Bertrand, Germaine est la mère de Jean, Christiane est la mère de Simone et de Paul, Simone est la mère de Benoît.

Attention à ne pas mettre de majuscules aux prénoms et aux prédicats et à ne pas oublier les points. Ceci fait,

- Traduire les questions suivantes sous forme de requêtes Prolog et vérifier les réponses :
 - Est-ce que Paul est un homme ?
 - Est-ce que Benoît est une femme ?
 - Qui est une femme ?
 - Qui est un homme ?
 - Est-ce que Marie est la mère de Sophie ?
 - Est-ce que Marie est la mère de Benoît ?
 - Qui est la mère de Jean ?
 - Quels sont les enfants de Paul ?
 - Quels sont les hommes qui sont pères ?
 - Quels sont les pères ?
- Définir les prédicats suivants :
 - `parent_de(X,Y)` : X est un parent de Y, en tant que père ou mère ;
 - `fils_de(X,Y)` : X est un fils de Y ;
 - `fille_de(X,Y)` : X est une fille de Y ;
 - `enfant_de(X,Y)` : X est un enfant de Y ;
 - `grand_pere_de(X,Y)` : X est un grand-père de Y ;
 - `grand_mere_de(X,Y)` : X est une grand-mère de Y ;
 - `frere_de(X,Y)` : X est un frère de Y (mêmes mère et père) ;
 - `soeur_de(X,Y)` : X est une sœur de Y (mêmes mère et père).
 - `demi-frere_de(X,Y)` : X est un demi-frère de Y (mêmes mère ou père, mais pas les deux) ;
 - `demi-soeur_de(X,Y)` : X est une demi-sœur de Y (mêmes mère ou père, mais pas les deux).

Qui est demi-frère de qui ? Qui est demi-sœur de qui ?

Exercice 4. Traduire en Prolog l'énoncé suivant :

- Marie aime le vin.
- Pierre aime tous ceux qui aiment le vin.
- Pierre est un voleur.
- Si quelqu'un est un voleur et aime quelque chose alors il le vole.

Qui vole quoi ?

Exercice 5. On considère l'ensemble suivant de faits sur Prolog :

```
sign(X,negatif) :- X < 0.
sign(X,zero) :- X == 0.
sign(X,positif) :- X > 0.
```

- Observez la réponse donnée par Prolog si vous lui posez la question suivante :
`?- sign(-1,Y).`
 Que se passe t'il et pourquoi Prolog ne termine pas l'exécution après avoir répondu `Y = negatif` ?
 Peut-on dire que l'ensemble de faits ci-dessus décrit un programme Prolog efficace ?

2. Utilisez le cut `!`, en l'ajoutant à l'ensemble de faits ci-dessus, pour rendre le programme Prolog plus efficace, sans changer sa sortie. Notamment, on voudrait que l'exécution termine immédiatement quand Prolog répond à chacune des questions suivantes :

```
?- sign(-1,Y).  
?- sign(0,Y).  
?- sign(1,Y).
```

Exercice 6. Prolog permet la gestion de *listes*, comme une succession d'éléments séparés par des virgules et entourés de crochets, par exemple :

```
[alain, alexis, frederic, safa, vincent]  
[1, alain, alexis, toto(X), Z, frederic, safa, vincent]  
[alain, [alexis, [frederic, [safa, [vincent[]]]]]]  
[]
```

On considère qu'une liste est toujours constituée d'une tête (premier élément) et d'une queue (la sous-liste restante) ; par exemple, la tête et la queue de la liste

```
[alain, alexis, frederic, safa, vincent]
```

sont respectivement `alain` et `[alexis, frederic, safa, vincent]`. Afin de décomposer commodément une liste en tête et queue, Prolog met à disposition le séparateur `|` (pipe) ; par exemple :

```
[X|Y] = [alain, alexis, frederic, safa, vincent]
```

renvoie :

```
X = alain  
Y = [alexis, frederic, safa, vincent]
```

Explorer une liste pour savoir si un élément est présent dans une liste se fait récursivement en deux lignes de programme :

```
element_de(X,[X,Q]).  
element_de(X,[T,Q]) :- element_de(X,Q).
```

Expérimenter ces deux lignes sur l'une des listes données en exemple, puis en vous inspirant de la façon dont elles sont écrites,

1. proposer un nouveau prédicat `elements_impairs_de(L,LI)` permettant d'extraire tous les éléments de rang impairs de la liste `L` dans la liste `LI` et tester sur la liste `L = [0, 1, 2, 3, 4, 5]` ;
2. proposer un prédicat permettant de produire la liste inversée d'une liste initiale.